



Oracle Bitmap Indexing

Resolving the ORA-08106 Rebuild Restriction

Management Technical Briefing

| Agenda & Learning Objectives

☰ **Introduction:** Role of Bitmap Indexes in Data Warehousing

✖ **The Error:** Analyzing the ORA-08106 Online Rebuild Failure

🔗 **Architecture:** Fragment-level Locking and Bit-to-Row Mapping

⚙️ **Best Practices:** Offline Maintenance and Parallelism Strategies

>_ **SQL Demo:** Step-by-Step Technical Validation

Understanding Bitmap Indexes

Low Cardinality Specialists

Bitmap indexes are designed specifically for columns with few distinct values (e.g., Gender, Marital Status, Region).

Unlike B-Tree indexes that store a list of RowIDs for each key, Bitmaps store a "bit string" where 1 indicates presence and 0 indicates absence.

$$\text{Efficiency} \propto \frac{1}{\text{Cardinality}}$$



The Conflict

Encountering ORA-08106

Why standard online maintenance patterns fail when applied to bitmap structures.

ORA-08106 Error Anatomy



The Syntax

Occurs when REBUILD
ONLINE is invoked on a
bitmap index.



The Block

Oracle raises ORA-08106
immediately during the
parsing phase.



The Warning

Bitmap indexes must be
rebuilt **offline** during
maintenance.

ORA-08106: cannot create/rebuild bitmap index online

| Architectural Constraint

Bit-to-Row Mapping

A single bitmap fragment covers a range of rows. In Oracle, a single index entry represents a continuous block of RowIDs.

This high compression ratio is the source of the limitation: you cannot journal changes to individual bits within a compressed fragment while that fragment is being rebuilt.

Fragment Management

Bitmaps are stored in fragments to handle table growth. Each fragment contains:

- Start RowID
- End RowID
- Bitmap Bitstring (compressed)

| Locking Impact Analysis



Note: Updating a single row in a bitmap-indexed table requires a lock on the entire bitmap fragment, significantly increasing contention during concurrent DML.

| Why "Online" is Prohibited



Journaling Complexity

Online rebuilds use a **Journal Table** to capture DML during the operation. For Bitmaps, capturing a DML change would require locking the fragment in the journal.

Because multiple rows map to one fragment, the merge process at the end of an online build would face insurmountable locking conflicts and CPU overhead for compression.

Solution: Offline Rebuild

- 📅 Schedule during low-activity windows.
- ⚡ Use **PARALLEL** to reduce downtime.
- ✓ Automatically invalidates and recreates the index structure.

```
ALTER INDEX idx_sales_reg  
REBUILD PARALLEL 4;
```



| Bulk Load Strategy

The Drop-and-Recreate Pattern

In Data Warehousing environments, the most efficient way to handle large bulk loads is to bypass indexing entirely during the load.

This avoids the massive overhead of updating bitmap fragments for every single row inserted.



Step 1: DROP the bitmap indexes.



Step 2: Perform the bulk data load.



Step 3: Gather statistics.



Step 4: CREATE BITMAP INDEX.

Technical Demo Results

Operation Step	SQL Command Used	Outcome / Status
1. Load Data	INSERT INTO sales_data ...	Success (Rows Committed)
2. Gather Stats	DBMS_STATS.GATHER_TABLE_STATS	Optimized (Histograms Set)
3. Online Rebuild	ALTER INDEX ... REBUILD ONLINE	FAILED (ORA-08106)
4. Offline Rebuild	ALTER INDEX ... REBUILD	Success (Index Valid)

Questions?

Managing Database Integrity and Performance

Technical Resource

Oracle Documentation
v19c/21c

Presenter Contact

Database Engineering Team

Image Sources



<https://www.techasoft.com/debug/img/oracle.png>

Source: www.techasoft.com



https://png.pngtree.com/background/20211215/original/pngtree-binary-data-light-effect-blue-abstract-background-picture-image_1466851.jpg

Source: pngtree.com



https://static.vecteezy.com/system/resources/previews/067/480/992/non_2x/3d-icon-bright-orange-padlock-securing-digital-symbolizing-data-protection-with-clean-transparent-background-isolated-png.png

Source: www.vecteezy.com



https://media.istockphoto.com/id/2230807736/photo/futuristic-ai-data-center-interior.jpg?s=612x612&w=0&k=20&c=E2s9c_8IA4fBpNmX4gzn2qLrne4mXW1jhYOf9Essem8=

Source: www.istockphoto.com